

questions-for-Simson.md

High level changes review, from Claude session "HVAC changes review - 27Jun2026". Involved detailed interaction with Claude but no human code review yet.

Topic #1: Temperature calcuations from multiple sensors

- Why does Area 51 ignore the FCU input? Presumably to get more human-relevant room temperatures, right? Any risk of pathological results if the wall sensors go dead/disconnected for whatever reason?
- Looks like most rooms are still using default. Deliberate? Or just not yet complete?

FCU	Inlet °C	Calculated °C	Set range °C	Sour
Restrooms/BOH	23.2	23.2	20.5–23.5	default (se
Area 51	21.3	21.7	19.9–22.9	7-source bl
Huddle Rooms	22.8	22.8	18.0–23.0	default
Sequoia	23.2	23.2	19.3–24.1	default
Eucalyptus	23.6	23.6	19.3–23.7	default
Greenhouse	26.0	26.0	18.9–24.1	default
Bamboo	26.0	26.0	20.0–23.0	default
Kitchen	25.6	25.6	20.1–23.1	default
Broadway North	24.4	24.4	18.0–24.2	self 1.0 (t
Broadway South	24.0	24.0	18.0–24.2	default



Topic #2: Rules engine rewrite

Potentially stale comment on line 33 of rules.py. Assumes that there are only kitchen and bathroom ERVs, since not guarded by code.

Topic #3: Use Flyway for migrations

Great!

A few small catches:

- There is a potential timing clash with cron. Better to stop the cron job while the migration runs. (per Claude; not verified). *Bead hvac-kag*
- CLAUDE.md needs to be updated

Other claude catches that I don't fully understand. Quoting here for reference. *Also in beads hvac-9ai, hvac-a9u, and hvac-7s5:*

↓↓↓↓↓↓↓↓ BEGIN CLAUDE QUOTE ↓↓↓↓↓↓↓↓↓

4. Forward-only; the DB backup is the entire rollback story. Flyway Community has no undo. A bad migration is recovered only by restoring the timestamped backup make deploy takes. That backup is good and present — but it's the single point of recovery, it's a full 138 MB file copy each deploy (disk growth in PROD_BACKUP_DIR, no pruning visible), and the copy itself races the live writer (a cp of an in-flight SQLite file without .backup/snapshot can capture a torn page). Consider sqlite3 .backup or VACUUM INTO instead of cp, plus backup retention.
5. V6's device_type backfill is a one-shot heuristic. device_type was inferred by name LIKE 'erv%' / ae200_device_id IS NOT NULL. It produced the right 10 FCU + 2 ERV today, but it's a point-in-time guess; correctness for any future device now depends entirely on app code setting the column, with no constraint enforcing it. A device added with a null device_type silently won't match device.erv/is_fcu_device (ties back to the ERV-generalization point in #2). Minor, but it's untyped data the rules engine trusts.
6. Flyway version pinned in CI but not on the prod host. The "pin CI install" commit pins the Linux CI Flyway. make deploy on slg1 uses whatever Flyway is installed there. Checksums are generally version-stable, but a mismatched version is a classic source of surprise validate failures. Worth pinning prod to the same version.

↑↑↑↑↑↑↑↑ END CLAUDE QUOTE ↑↑↑↑↑↑↑↑

Topic #4: Full HVAC command set including DRY and AUTO

Problem detected by Claude; not yet confirmed:

set_set_temp() can only write one value. This is not well-defined when in AUTO mode, which requires two values.

Topic #5: Hubitat simulator

Unlike the AE200 simulator, this one is read-only. (Claude is concerned; I'm not sure that it is something that we need. Flagged as P4 bead hva-bqb to at least document)

Two more issues flagged by Claude. Quoting literally here because they may require discussion with Simson. *Also captured as beads hvac-b0c and hvac-sep:*

↓↓↓↓↓↓↓↓ BEGIN CLAUDE QUOTE ↓↓↓↓↓↓↓↓

A) Inconsistent simulator env-var parsing — a genuine footgun

The three simulator flags don't agree on what "on" means:

Flag	Parsing
HUBITAT_SIMULATOR	in {1, true, yes, on} (robust)
AE200_SIMULATOR	os.getenv(...) — any non-empty string is truthy
AIRTHINGS_SIMULATOR	os.getenv(...) — any non-empty string is truthy

The new Hubitat one is the correct implementation. The two older ones are fragile: AE200_SIMULATOR=0 or AE200_SIMULATOR=false — the natural way someone would try to disable it — actually enables the simulator, because "0" is a non-empty string. make local-dev (sets =1) and make local-live-dev (sets = empty) both happen to work, so it's hidden today. But this is exactly the flag that decides whether commands reach real HVAC hardware, so a wrong-way-round simulator flag is the highest-stakes place to have this ambiguity. The fix is trivial and the new Hubitat helper is the template — AE200/AIRTHINGS should adopt the same TRUE_ENV_VALUES check.

B) Two Hubitat fixtures — and the one the simulator serves is untested

- Simulator serves: app/test_data/hubitat_get_devices.json — 17 devices, attributes as a dict ({"temperature": "19.4", "humidity": "15", ...}).
- Tests validate: etc/sample_hubitat.json — a different file, with a different shape (the extraction test checks numeric conversion against it).
- The simulator fixture is referenced by zero tests.

So the JSON that make local-dev actually feeds through `extract_temperatures()` / `update_from_hubitat()` is never exercised by the test suite. If its shape drifts from what the parser expects, tests stay green while local-dev silently produces wrong or empty data — the worst kind of gap, because the simulator's whole purpose is to be a faithful stand-in. Given the two fixtures already have visibly different attributes structures, "are they actually interchangeable?" is an open question the tests don't answer. Lowest-effort fix: point the existing extraction test at the simulator fixture too (or make it the single shared fixture).

↑↑↑↑↑↑↑↑ END CLAUDE QUOTE ↑↑↑↑↑↑↑↑

Topic #6: Web UI Screenshot pipeline

I'm not sure about this one. It's cute and powerful, but add a lot of clutter to the history. Let's discuss.

Topic #7: Air quality coloring

I like this a lot.

Claude detects two issues that I don't know enough to address: (*beads hvac-o33 and hvac-hem*)

↓↓↓↓↓↓↓↓ BEGIN CLAUDE QUOTE ↓↓↓↓↓↓↓↓↓

A) PM1 reuses PM2.5 thresholds — not standards-based. pm1 has identical breakpoints to pm25 (fair >9, poor >35.4). But PM1 is a finer size fraction fully contained in PM2.5, so $PM1 \leq PM2.5$ always, and there is no recognized health standard for PM1 (the 9 / 35.4 figures are EPA PM2.5 numbers). Net effect: PM1 cells will almost never reach "poor," and when they're colored at all the thresholds are essentially borrowed/arbitrary. Either drop PM1 coloring, or document that it's a heuristic reuse of PM2.5, not a standard.

B) Humidity has no "fair" band on the dry side — a hard cliff. High humidity grades good→fair (>50)→poor (>60), but the low side jumps straight good→poor at 30 with no fair buffer: 31% = good, 29% = poor. The asymmetry is a minor UX inconsistency; a fairBelow (e.g. 30–40 = fair) would mirror the high side. The classification code doesn't even support fairBelow today, so it's a small feature, not just a threshold tweak.

(For calibration vs. recognized guidance: CO₂ 1000 ppm, radon 150 Bq/m³ ≈ EPA action level, PM2.5 9/35.4 ≈ current EPA — all reasonable. VOC "good ≤500" is a bit more lenient than Airthings' own ≤250, but defensible.)

↑↑↑↑↑↑↑↑ END CLAUDE QUOTE ↑↑↑↑↑↑↑↑

Topic #8: Pydantic data contracts

Excellent. Claude flagged one issue (*bead hvac_obp*). Is it right, or are you reserving this for expansion?

↓↓↓↓↓↓↓↓ BEGIN CLAUDE QUOTE ↓↓↓↓↓↓↓↓↓

The concern worth flagging — the extra policy is inconsistent, and backwards where it matters most:

Model kind	extra p
StatusPayload, CommandResponse, DeviceStatus	allow (intentional vend
WeatherData	forbid
All *Control request bodies	default = ignore

The HVAC-control request models use Pydantic's default extra="ignore", so a client that sends a misspelled or wrong field name gets a 200 with an unintended action, not a 400. Example: POST /set_temp {"device_id": 12, "set_temp": 21} — the real field is set_temp_c, so set_temp is silently dropped and set_temp_c is missing/wrong. For endpoints that command real hardware, silently ignoring unknown fields is exactly where you'd want strict extra="forbid" to catch the client bug. Meanwhile WeatherData — a response/parse model, the place where leniency is usually safer — is the one set to forbid. So the strictness is applied roughly inversely to where it helps.

The fix is cheap and high-value: set extra="forbid" on the *Control request models (catch typo'd commands), and confirm WeatherData's forbid is intentional (if it ever parses an external weather payload, an added upstream field would throw).

↑↑↑↑↑↑↑↑ END CLAUDE QUOTE ↑↑↑↑↑↑↑↑

Topic #9: Tech-debt review

Claude notes that your (Copilot's?) Code review caught many of the same issues as above. It suggested that we use the same tracking tools. It's right, of course, but I'm very happy with Beads and would guess that you are equally happy with GitHub. At our current scale, I think we can live with this conflict. What do you think?

Just for completeness, here are the overlaps that Claude noticed:

↓↓↓↓↓↓↓↓ BEGIN CLAUDE QUOTE ↓↓↓↓↓↓↓↓↓

The tech-debt review is genuinely well-organized: an executive summary, 14 ranked items grouped (Highest Priority → Reliability/Ops → Architecture → UI Debt → Decision-Dependent), a GitHub-issue triage section, and a concrete "Suggested Next Sequence." The headline items:

1. Rewrite the rules system around typed inputs/returned settings (largely done — theme #2)
2. Productize the map — build /map as a real feature, not a static prototype
3. Stabilize operations — service ownership, deploy, backups, credentials
4. Preserve diagnostic status_json across retention cleanup
5. Reduce route/data-layer coupling (split app/db.py)

What's striking: this doc independently corroborates several issues I flagged while reviewing #1–#8. It's effectively the maintainers' own confirmation of the same problems:

- Item #2 "Productize the map" explicitly calls rooms/map a "static prototype," not a real feature — which is exactly why I found the rooms table empty (0 rows) in production back in theme #1. The empty table is known prototype debt, not an oversight.
- Item #4 "Backups and secrets" says there's "no complete documented backup/restore procedure" and recommends doc/operations.md + a restore drill — directly overlapping my P2 bead hvac-9ai (cp-based backup, no documented restore).
- Item #5 "Preserve status JSON" confirms combine_temp_measurements() drops status_json during run-length compression — related to the audit-log gaps I noted in theme #4.

- Item #6 "Split app/db.py" confirms the ~1,500-line db.py is recognized debt, and flags that it imports Flask request and integration modules (coupling) — corroborating my "look closely at db.py" note from theme #1.

So my independent findings and the project's own audit converge — that's a good validation signal on both sides.

One process concern worth raising: the tech-debt items reference GitHub issues (#42, #56, #127, ...), but this project mandates beads (bd) for all task tracking ("never use markdown TODOs"; AGENTS.md/CLAUDE.md). So the project's most important roadmap currently lives in a markdown doc + GitHub issues, outside the mandated tracker. The beads I've been filing this session (rules, backups, db split, etc.) may overlap or duplicate those GitHub issues. There's a real question of which tracker is authoritative — and if it's beads, this roadmap should be imported so it's not split-brained.

↑↑↑↑↑↑↑↑ END CLAUDE QUOTE ↑↑↑↑↑↑↑↑

Here is Claude's summary of the issues it detected now. We should merge and compare with tech-debt.md

↓↓↓↓↓↓↓↓↓ BEGIN CLAUDE QUOTE ↓↓↓↓↓↓↓↓↓↓

By priority

P1 — act before next deploy (1)

- hvac-kag (bug) — Flyway migrations run against a live, cron-written 138 MB SQLite DB without quiescing the every-minute runner. V5 is a full-table UPDATE over 778k rows; collision → SQLITE_BUSY or a half-applied migration needing flyway repair.

P2 — real, low-effort (2)

- hvac-u5b — CLAUDE.md still says "no migration framework / manual ALTER," actively steering contributors to bypass Flyway and hand-edit the now-generated schema.sql.
- hvac-9ai — Flyway is forward-only; rollback relies solely on a cp-based backup (torn-page risk, no retention, all-or-nothing restore).

P3 — substantive (8)

- hvac-7s5 — Flyway version pinned in CI but not on the prod host (slg1).
- hvac-a9u — device_type backfill (V6) is a one-shot heuristic; new devices with null device_type silently escape the rules engine.
- hvac-1pv — Most HVAC commands (set-temp/drive/fan) log to changelog with a blank comment; the highest-frequency actions are the least self-describing.

- hvac-zsl (bug) — AUTO mode's dual setpoints/band are read-only; /set_temp writes only the single SetTemp, so AUTO is reachable but not fully drivable.
- hvac-b0c (bug) — Simulator env flags parse inconsistently: AE200_SIMULATOR=0/false enables the simulator (the flag that decides real-vs-fake hardware).
- hvac-o33 — PM1 air-quality coloring reuses PM2.5 thresholds (no standards basis).
- hvac-obp (bug) — *Control request models use extra='ignore', so HVAC endpoints silently accept typo'd/unknown fields with a 200.
- hvac-7mx — DISCUSSION (David + Simson): align on a frontend testing strategy (Playwright reintroduced two ways vs. the "avoid Playwright" direction; CI-skipped browser tests; three fake-data sources).

P4 — minor / nice-to-have (3)

- hvac-bqb — Hubitat simulator is read-only; command paths still hit real hardware in local-dev.
- hvac-sep — Hubitat simulator fixture is served in local-dev but exercised by no test.
- hvac-hem — Humidity coloring has no "fair" band on the dry side (hard good→poor cliff at 30%).

By theme reviewed

Theme	Beads
#1 Calculated temps/rooms	(no bugs – but rooms confirmed unused; corro
#2 Rules engine rewrite	hvac-a9u (ERV/device_type generalization tra
#3 Flyway migrations	hvac-kag, hvac-u5b, hvac-9ai, hvac-7s5
#4 AE-200/FCU controls	hvac-1pv, hvac-zsl
#5 Hubitat simulator	hvac-b0c, hvac-bqb, hvac-sep
#6 Screenshot pipeline	hvac-7mx
#7 AQ coloring	hvac-o33, hvac-hem
#8 Pydantic contracts	hvac-obp
#9 Tech-debt doc	(corroborates hvac-9ai, hvac-1pv; raised tra

not filed)

↑↑↑↑↑↑↑↑ END CLAUDE QUOTE ↑↑↑↑↑↑↑↑

